

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS	
Formato: Guia de Practica de Laboratorio / Talleres / Centros de Simulacion		
Aprobacion: 2022/03/01	Codigo: GUIA-PRLE-001	Pagina: 1

INFORME DE LABORATORIO

INFORMACION BASICA					
ASIGNATURA:	FISICA COMPUTACIONAL				
TITULO DE LA PRACTICA:	Laboratorio 2 - Grupo A				
NUMERO DE LA PRACTICA:	02	ANO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACION:	05/05/2026	HORA DE PRESENTACION:	11:59:00		
INTEGRANTE(s): - Noa Camino Yenaro				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Prof. Edwin Agapito Llamoca Requena					

RESULTADOS Y PRUEBAS
SOLUCION DE LOS EJERCICIOS PROPUESTOS 0.1 Ejercicio 1: Circunferencia de radio 3 Problema: Graficar una circunferencia con radio $r = 3$ cuyo centro esta en el origen. Implementacion: Se simulo el movimiento con paso h y se grafico la trayectoria en el plano XY. La configuracion mantiene el centro en el origen y el radio objetivo. Codigo fuente: <pre>import math from pathlib import Path import matplotlib.pyplot as plt import numpy as np plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8}) plt.figure(figsize=(12, 10)) h = 0.01 k = 0.1 tfin = 50 def ax(x, y): r = math.sqrt(x**2 + y**2) return -x / r**3 def ay(x, y): r = math.sqrt(x**2 + y**2) return -y / r**3 for vx0 in np.arange(math.sqrt(1 / 3), 1.5, k): vx = vx0 vy = 0.0 x = 0.0 y = -3.0 n = 0</pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS	
Formato: Guia de Practica de Laboratorio / Talleres / Centros de Simulacion		
Aprobacion: 2022/03/01	Codigo: GUIA-PRLE-001	Pagina: 2

```

px = [x]
py = [y]

for _t in np.arange(0, tfin, h):
    n += 1
    vx = vx + ax(x, y) * h
    vy = vy + ay(x, y) * h
    x = x + vx * h
    y = y + vy * h
    px.append(x)
    py.append(y)

plt.plot(px, py)

plt.gca().set_aspect("equal", adjustable="box")
output_path = Path(__file__).resolve().parent.parent / "assets" / "1.png"
plt.savefig(output_path, dpi=300, bbox_inches="tight")
plt.close()

```

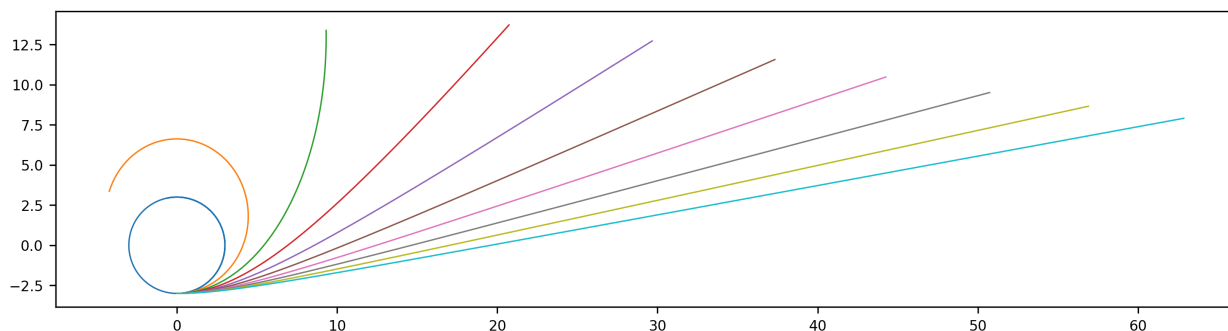


Figura 1: Circunferencia de radio 3.

0.2 Ejercicio 2: Omision grafica en $r \leq 3$

Problema: Cuando la trayectoria llegue a $r = \sqrt{x^2 + y^2} \leq 3$ no debe graficar, pero la simulacion debe continuar.

Implementacion: Se evalua el radio en cada paso y se registran nan en los puntos donde $r \leq 3$ para que la grafica omita el trazo, manteniendo la evolucion temporal.

Codigo fuente:

```

import math
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8})
plt.figure(figsize=(12, 10))

h = 0.01
k = 0.05
tfin = 200

def ax(x, y):
    r = math.sqrt(x**2 + y**2)
    return -x / r**3

def ay(x, y):
    r = math.sqrt(x**2 + y**2)

```

```

return -y / r**3

for vx0 in np.arange(math.sqrt(1 / 3), 1.5, k):
    vx = vx0
    vy = 0.0
    x = 0.0
    y = -3.0
    n = 0

    px = []
    py = []

    for _t in np.arange(0, tfin, h):
        n += 1
        vx = vx + ax(x, y) * h
        vy = vy + ay(x, y) * h
        x = x + vx * h
        y = y + vy * h
        if math.sqrt(x**2 + y**2) > 3:
            px.append(x)
            py.append(y)
        else:
            px.append(np.nan)
            py.append(np.nan)

    plt.plot(px, py)

plt.gca().set_aspect("equal", adjustable="box")
output_path = Path(__file__).resolve().parent.parent / "assets" / "2.png"
plt.savefig(output_path, dpi=300, bbox_inches="tight")
plt.close()

```

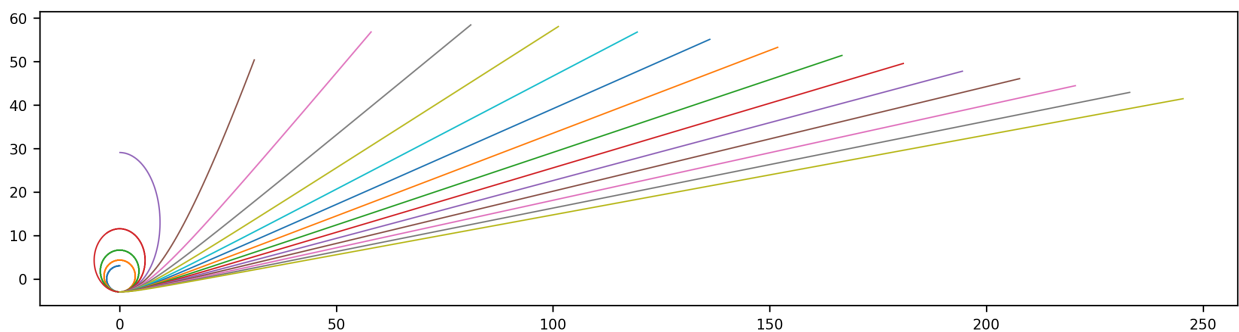


Figura 2: Trayectoria con omision en $r \leq 3$.

0.3 Ejercicio 3: Parabolas, elipses e hipernbolas

Problema: Graficar en una misma ventana dos parabolas, dos elipses y dos hipernbolas.

Implementacion: Se clasificaron trayectorias segun la energia y el tipo de orbita. Se seleccionaron dos ejemplos por tipo y se presentaron en un solo plano.

Codigo fuente:

```

import math
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8})
plt.figure(figsize=(12, 5))

```



```
# Parámetros
h = 0.05
k = 0.1
tfin = 450

# Funciones
def axr(x, y):
    return -x / (x**2 + y**2)

def ayr(x, y):
    return -y / (x**2 + y**2)

def ax(x, y):
    return -x / (x**2 + y**2) ** (3 / 2)

def ay(x, y):
    return -y / (x**2 + y**2) ** (3 / 2)

# ----- Primera trayectoria -----
vx = 1
vy = 0.0
x = 0
y = 3

px = [x]
py = [y]

t_values = np.arange(0, tfin, h)

for t in t_values:
    vx += axr(x, y) * h
    vy += ayr(x, y) * h
    x += vx * h
    y += vy * h
    px.append(x)
    py.append(y)

plt.plot(px, py)

# ----- Clasificación de trayectorias -----
crash_x1, crash_y1 = [], []
crash_x2, crash_y2 = [], []
orbit_x1, orbit_y1 = [], []
orbit_x2, orbit_y2 = [], []
exit_x1, exit_y1 = [], []
exit_x2, exit_y2 = [], []

ix, iy = 0, -4

# 6 valores fijos
vx_values = [0.2, 0.25, math.sqrt(1 / 3) + 0.001, math.sqrt(1 / 3) + 0.02, 2.5, 3.5]

for vx0 in vx_values:
    vx = vx0
    vy = 0.0
    x = ix
    y = iy

    px = [x]
    py = [y]

    for t in t_values:
        vx += ax(x, y) * h
        vy += ay(x, y) * h
        x += vx * h
        y += vy * h
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS	
Formato: Guia de Practica de Laboratorio / Talleres / Centros de Simulacion		
Aprobacion: 2022/03/01	Codigo: GUIA-PRLE-001	Pagina: 5

```

    if np.sqrt(x**2 + y**2) <= 3:
        break

    px.append(x)
    py.append(y)

r = np.sqrt(x**2 + y**2)

if r <= 3:
    if not crash_x1:
        crash_x1, crash_y1 = px, py
    elif not crash_x2:
        crash_x2, crash_y2 = px, py

elif r <= 20:
    if not orbit_x1:
        orbit_x1, orbit_y1 = px, py
    elif not orbit_x2:
        orbit_x2, orbit_y2 = px, py

else:
    if not exit_x1:
        exit_x1, exit_y1 = px, py
    elif not exit_x2:
        exit_x2, exit_y2 = px, py

# ----- Gráficas -----
plt.plot(crash_x1, crash_y1)
plt.plot(crash_x2, crash_y2)
plt.plot(orbit_x1, orbit_y1)
plt.plot(orbit_x2, orbit_y2)
plt.plot(exit_x1, exit_y1)
plt.plot(exit_x2, exit_y2)

plt.axis("equal")
plt.xlim([-10, 50])
plt.ylim([-6, 12])

a = 5
b = 3

x_vals = np.linspace(-50, 50, 1000)

output_path = Path(__file__).resolve().parent.parent / "assets" / "3.png"
plt.savefig(output_path, dpi=300, bbox_inches="tight")
plt.close()

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS	
Formato: Guia de Practica de Laboratorio / Talleres / Centros de Simulacion		
Aprobacion: 2022/03/01	Codigo: GUIA-PRLE-001	Pagina: 6

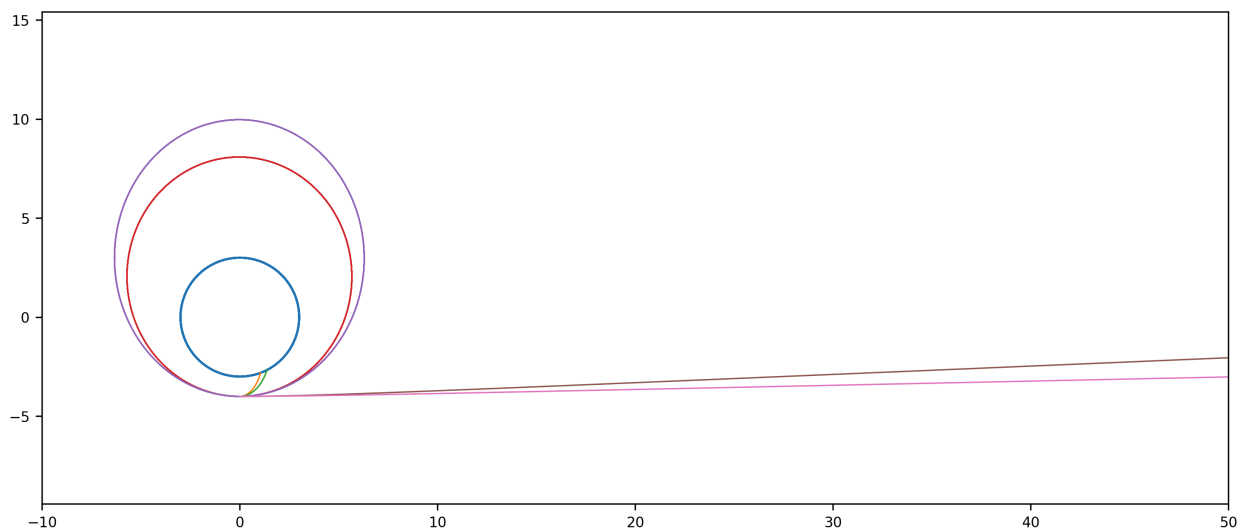


Figura 3: Dos parabolas, dos elipses y dos hipernbolas.

0.4 Ejercicio 4: Repeticion con $v_y < 0$

Problema: Repetir el paso anterior para $v_y < 0$ y notar el efecto en las trayectorias.

Implementacion: Se modifico la condicion inicial con velocidad vertical negativa y se repitio la clasificacion para observar la inclinacion y el cambio en el tipo de trayectoria.

Codigo fuente:

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8})
plt.figure(figsize=(12, 8))

# Parámetros
h = 0.05
k = 0.1
tfin = 1450

# Funciones
def axr(x, y):
    return -x / (x**2 + y**2)

def ayr(x, y):
    return -y / (x**2 + y**2)

def ax(x, y):
    return -x / (x**2 + y**2) ** (3 / 2)

def ay(x, y):
    return -y / (x**2 + y**2) ** (3 / 2)

# ----- Primera trayectoria -----
vx = 1
vy = 0.0
```

```
x = 0
y = 3

px = [x]
py = [y]

t_values = np.arange(0, tfin, h)

for t in t_values:
    vx += axr(x, y) * h
    vy += ayr(x, y) * h
    x += vx * h
    y += vy * h
    px.append(x)
    py.append(y)

plt.plot(px, py)

# ----- Clasificación de trayectorias -----
crash_x1, crash_y1 = [], []
crash_x2, crash_y2 = [], []
orbit_x1, orbit_y1 = [], []
orbit_x2, orbit_y2 = [], []
exit_x1, exit_y1 = [], []
exit_x2, exit_y2 = [], []

ix, iy = 0, -4

# 6 valores fijos
vx_values = [
    0.6,
]

for vx0 in vx_values:
    vx = vx0
    vy = -0.1
    x = ix
    y = iy

    px = [x]
    py = [y]

    for t in t_values:
        vx += ax(x, y) * h
        vy += ay(x, y) * h
        x += vx * h
        y += vy * h

        if np.sqrt(x**2 + y**2) <= 3:
            break

        px.append(x)
        py.append(y)

    r = np.sqrt(x**2 + y**2)

    if r <= 3:
        if not crash_x1:
            crash_x1, crash_y1 = px, py
        elif not crash_x2:
            crash_x2, crash_y2 = px, py

    elif r <= 20:
        if not orbit_x1:
            orbit_x1, orbit_y1 = px, py
        elif not orbit_x2:
            orbit_x2, orbit_y2 = px, py

    else:
        if not exit_x1:
            exit_x1, exit_y1 = px, py
        elif not exit_x2:
```

```
exit_x2, exit_y2 = px, py

# ----- Gráficas -----
plt.plot(crash_x1, crash_y1)
plt.plot(crash_x2, crash_y2)
plt.plot(orbit_x1, orbit_y1)
plt.plot(orbit_x2, orbit_y2)
plt.plot(exit_x1, exit_y1)
plt.plot(exit_x2, exit_y2)

plt.axis("equal")
plt.xlim([-10, 20])
plt.ylim([-8, 12])

a = 5
b = 3

x_vals = np.linspace(-50, 50, 1000)

output_path = Path(__file__).resolve().parent.parent / "assets" / "4.png"
plt.savefig(output_path, dpi=300, bbox_inches="tight")
plt.close()
```

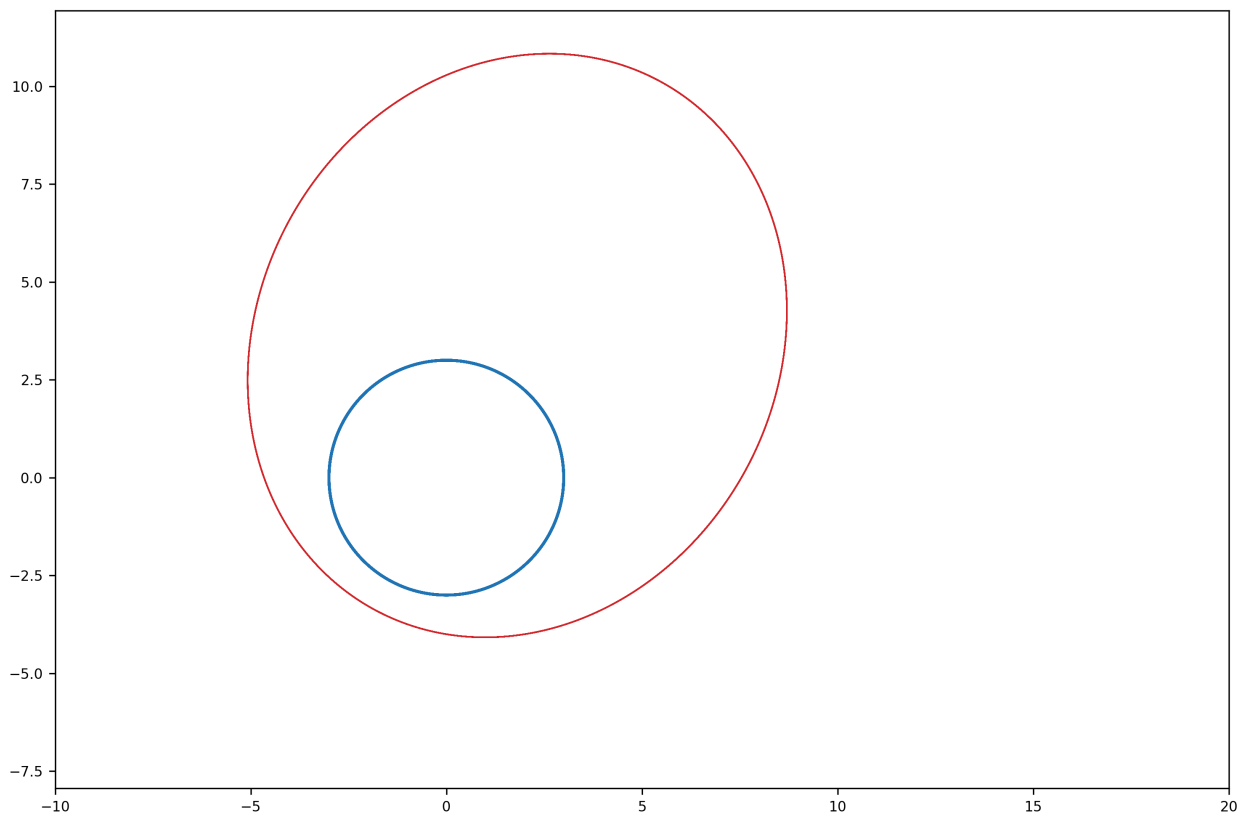


Figura 4: Trayectorias con $v_y < 0$.

CONCLUSIONES Y RECOMENDACIONES

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS	
Formato: Guia de Practica de Laboratorio / Talleres / Centros de Simulacion		
Aprobacion: 2022/03/01	Codigo: GUIA-PRLE-001	Pagina: 9

CONCLUSIONES

1. Se logro representar la circunferencia de radio 3 y controlar el trazo cuando el radio cae dentro del umbral, manteniendo la simulacion continua.
2. La clasificacion de trayectorias permite distinguir parabolas, elipses e hipernbolas en una sola figura, facilitando la comparacion visual.
3. Al imponer $v_y < 0$ se observa un cambio claro en la forma e inclinacion de las trayectorias, evidenciando la influencia de las condiciones iniciales.

RECOMENDACIONES

1. Mantener documentadas las condiciones iniciales y el criterio de clasificacion para reproducir las trayectorias y comparar resultados entre ejecuciones.